Darren Lee
CSE13S

Week 9 Notes

5/24/21

Hash

- Means chopping things up and mixing them around
- We take a key (a string) and make a number from it
- The resulting number should be uniformly distributed

Problem

- We want to use this to find records based on a key
- Queries we should be able to make efficiently:

1. Insert a key and its corresponding value
2. Search for a key and fetch the corresponding value
3. Delete a key along with its corresponding value

Data structures that can be used to solve the problem?

- Expected time in arrays and linked lists:
    - Linear search: look-ups in unsorted arrays take O(n)
    - Binary search: look-ups in sorted arrays take O(log(n))
    - Linked list look-ups take O(n)
- Direct address table is a key-value store where the key is used as an index to the value:
    - Random access arrays: searching in O(1) time
    - Limitations:
        - Need prior knowledge of maximum key value
        - Wastage of memory space if there is a significant difference between total records and preset maximum table size

Hashing

- The idea behind hashing or scatter storage is to scramble up aspects of the key and use this partial information for searching
- Improvement over Direct Address Table:
    - Uses a function that hashes a larger fixed key into a smaller number
        - Less than or equal to the size of the table
    - Uses the smaller number as index of the value
- With hashing we get O(1) search time on average (under reasonable assumptions) and O(n) in worst case

Hash tables

- An unordered collection of key-value pairs
- Offers efficient lookup, insert, and delete operations
    - Neither arrays nor linked lists achieve all of these
- The Python dictionary is a hash table. Example:
    - `my_dict = {1:'Ron',2:'Bob'}`
    - Key 1 is associated with the name 'Ron' and key 2 is associated with the name 'Bob'.
-

Hash functions

- A function that hashes keys to generate indices for each value
    - For a hash table with m slots
        - $h(key) \rightarrow index \in [0, m-1]$
    - Characteristics of a good hash function are determined by the data being hashed:
        - Function uses all input data
        - Computation is fast
        - Minimizes hash collisions:
            - Uniformly distributes data across entire set of possible hash values
            - Generates very different hash values for similar strings
    - Types of hash functions
        - Division based
        - Multiplication based
        - String-valued keys

Hash functions: division based method

- Most simple method for hashing is the remainder modulo method
- Formula: h(K)=K mod M
    - K is the key
    - M is the table size

Hash functions: string-valued key

- Sometimes you want to use string-valued keys for a hash table
- Basi approach:
    - Tak characters from string-valued key
    - Compute an integer using an appropriate method
    - Integer modulo the table size
- Can we just add the letters?
    - Addition commutes
    - No, the number of possibilities is too small
- Can we just multiply them?
    - No multiplication also commutes
- Creating a good hash function is not easy

Example of a hash table

- ☐ Begin with an empty hash table
- ☐ Store numbers with their respective keys

- {91:54, 38:26, 47:93, 56:17, 2:77, 82: 31}
  - Hash function $h(key) = key \pmod{11}$
    - Remainder modulo method: key mod table size.
    - Remainder returned is an index whose value is between 0–10.
    - Values are stored in the index corresponding to each key.

0	1	2	3	4	5	6	7	8	9	10
None	None	None	None	None	None	None	None	None	None	None

$(h(item)=item\%11)$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

Load factor

- Number of slots filled out of total slots in a hash table
- In this example, it is denoted by $\lambda = \frac{6}{11}$.

0	1	2	3	4	5	6	7	8	9	10
None	None	None	None	None	None	None	None	None	None	None

$(h(item)=item\%11)$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

Birthday Paradox

- Let's say you are in a room with just 23 people, there's a 50-50 chance of at least 2 people having the same birthday
  - Counting pairs: with 23 people we have $\binom{23}{2} = 253$ pairs
  - The chance of 2 people having different birthdays is $1 - \frac{1}{365} = \frac{364}{365} = 0.997260$.
  - The probability of *all* 253 pairs having different birthdays is $p = \left(\frac{364}{365}\right)^{253} = 0.4995$.
  - Therefore, the probability that we find a pair with the same birthday is $1 - 0.4995 = 0.5005$.
  - If you change the number of people (n) in the room the probability changes such that

$$p(n) = 1 - \left(\frac{364}{365}\right)^{n \times \frac{n-1}{2}}$$

  - This applies to hash functions as well.

Hash collisions

- Occurs when two pieces of data have the same hash value
- Assume that h(x) is a hash function

- We have 2 keys K_i and K_j such that $K_i \neq K_j$.
  - A collision means $h(K_i) = h(K_j)$.
- Good hash functions, given a proper range of possible hash values, should rarely result in collisions.
- Collisions are unavoidable if the number of keys is greater than the number of hash values.
  - The cardinality of the domain of the hash function is larger than its range.

Tackling collisions

- Open addressing allows elements to move from their preferred position to other positions
  - Linear probing
  - Quadratic probing
  - Double hashing
- Chaining using linked lists

Linear probing

- Move sequentially through the hash table slots until you encounter the next empty slot
- For a table t os fsize SIZE, if we want to store an element x with hash index i at location of t[i], and that position is already occupied
  - Try to store it in location t[(i+1))%SIZE]
- Where index=(i+1)%SIZE
- If that position is occupied, we try
- t[(i+2)%SIZE0] and so on
- Until we find an index, such that either t[index[=x, or t[index]=NULL
- This is for the find() , add(), and the remove() functions

Quadratic probing

- Square the sequence of hash values when deciding how far to look for the next slot
  - Do this if the initial probe fails
- Front eh previous code: if we want to store an element x with hash index i in the table location of t[o] and that position is already occupied
  - Try to store it in location t[(ixi)%SIZE]
  - If that position is occupied, we try t[(i+1)x(i+!)%SIZE] and so on
- To guarantee that quadratic probing hits every single available spot eventually
  - Has table size must be a prime number
  - Hash table must never be more than half full

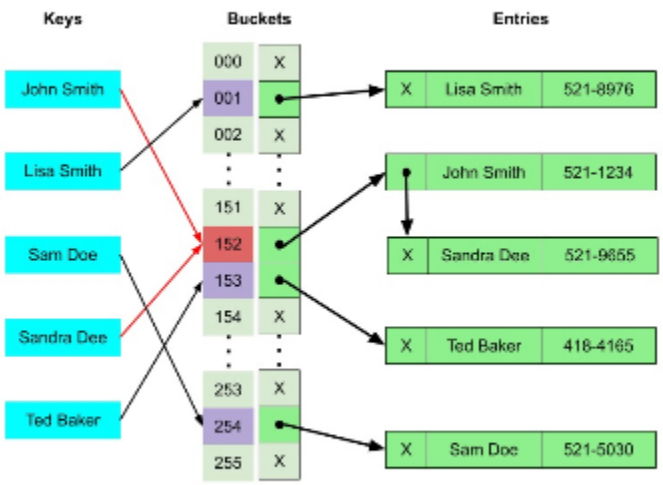
Double hashing

- Like linear and quadratic probing
- Uses a hash function to determine the $(i+1)_{st}$ hash location in table T containing n elements
- Load factor $\lambda = \frac{n}{|T|}$

- For example given hash functions h1,h2,. For a key
    - First, we check the location of h1(key)
    - If its not empty, we calculate h2(key)
    - $T(i, key) = (h_1(key) + i \times h_2(key)) \bmod |T|$

Chaining using Linked lists

- Each slot contains a link to a singly linked list
    - Key-value pairs with the same hash
- Look-up algorithm searches through the list to find matching key
- Initially table slots contain nulls
- List is being created, when value with the certain hash is added for the first time



-

Performance analysis of chaining

- m=number of slots in hash table
- n=number of keys to be inserted in hash table
- Load factor lambda=n/m
- Expected time to search=O(!+lambda)
- Expected time to insert/delete=O(!+lambda)
- Time complexity of search insert and delete is O(1) if alpha is O(1)

Open addressing vs chaining

- Open addressing is computationally intensive compared to chaining
- Dynamic nature: in chaining hash table is never filled up as we can add more elements to the chain
- Chaining is less sensitive to load factor and hash function
- Cache performance:
    - Chaining: bad as keys are stored using linked list
    - open addressing: good as everything stored in the same table
- Space:
    - Chaining uses extra space for links and parts of the hash table is never used

    - No links in open addressing and a slot can be used even if an input isn't mapped to it

The Hash Table ADT

- Figure 1 shows a general hash table ADT
    - Keytype and infotype are void * type so we can pass any type we need for our problem
    - Based on the type of key, the hash function hash_code() will change
- Figure 2 shows a hash table declaration for the problem on slide 12
    - Hash table of size 11: as we discussed before the hash table size should be a prime number
    - Both th value and the key are unsigned 32_bit integers
    - The hash function used here is the remainder method that takes a 32-bit integer as a key and returns an index in the table between 0-10
- Each data item consists of a key-value pair

Search in a hash table

- Searching using linear probing to avoid hash collisions
    - When an index is used, you will check the next index, index+!
    - This type of search wraps around the hash table and continues from the start of the array

Insertion

- Since traversin the hash table is done using linear probing
- Sequentially move forward through the indices

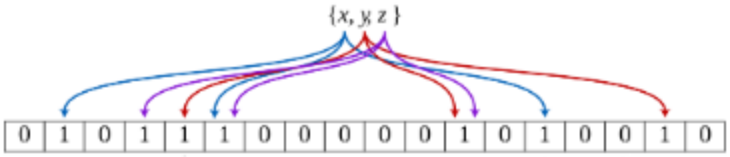
Deletion

- This function takes an input a key value pair and itf the contents of a hash table index matches the key, deletes the corresponding item
- Traverses forward through the hash table using linear probing

Bloom filters

- A data structure designed to tell you whether an elements i present in a set
- Internally structured like a bit array
- Two operations: add and search/query
- Assume bloom filter on the right has k=3 hash functions
- Pass elements (x,y,z) through 3 hash functions and set the corresponding bits in the bloom filter

Assume that the K hash functions are $h_1, h_2, and\ h_3$.



$h_1(x), h_2(x), and\ h_3(x)$ sets bits 1, 5 and 13.

$h_1(y), h_2(y), and\ h_3(y)$ sets bits 4, 11 and 16.

$h_1(z), h_2(z), and\ h_3(z)$ sets bits 3, 5 and 11.

-

Why bloom filters?

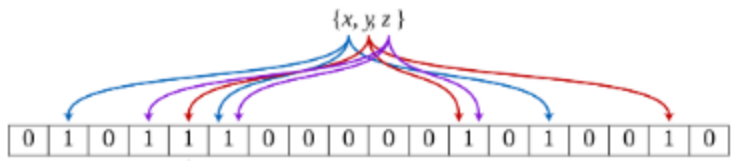
- Developed by Burton Howard Bloom in 1970
- Bloom filters are fast and memory efficient
- Called filters
    - Used as a first pass to filter out segments of datasets that do not match a query
- Related to has tables:
    - Can be used to efficiently check if a key exist in the hash table

False positives

- A bloom filter query return “possibly in a set” or “definitely not in set”

False positives:

- When passed through he same 3 hash function 2 bits are set by 2 different elements from the ste
- Bit 5 was set by x and z, bit 11 was set by y and z



$h_1(x), h_2(x), and\ h_3(x)$ sets bits 1, 5 and 13.

$h_1(y), h_2(y), and\ h_3(y)$ sets bits 4, 11 and 16.

$h_1(z), h_2(z), and\ h_3(z)$ sets bits 3, 5 and 11.

-

Probability of false positives

- Assume that a hash function selects each array position with equal probability
  - M is the number of bit in the array
  - K is the number of hash functions
  - N is the number of inserted elements
- $p = (1 - e^{-\frac{K \times N}{M}})^K$
- The more items per Bloom filter, the higher the chances of having false positives
- Increasing the number of hash functions can reduce false positives

Time and space complexity

- Seize of bloom filter is M bits and K hash functions
  - Insertion and search will both take O(k) time
- The space of the actual data structure (what holds the data) is simply O(M)

Other application of hash functions: Cryptographic Hash

- Message integrity:
  - Alice sends BOb a message, m | h where the message is concatenated with the message's hash
  - Bob computes the hash over the message and compares the received hash and the calculated hash
  - If they are different, the message has been altered
- Digital signatures
  - Alice signs a message with her private key
  - Bob can verify Alice's authorship with her public key
  - Inefficient to sign the entire message. Better to sign the message's hash value

Cryptography based hash function

- An ideal cryptographic hash function guarantees the following properties
  - Deterministic: same message result in the same hash
  - Quick to compute the hash for any given message

    - Infeasible to construct a message that yields a given hash value
    - Infeasible to find two different messages with the same hash value
    - Avalanche effect - a small change to the message should produce a hash value that appears uncorrelated with the old hash

Popular cryptographic hash functions

- md5 -broken
- Sha1-broken
- Sha256
- Sha512
- Sha3

Summary

- Hash functions
    - Map data of arbitrary size to fixed size values
    - Aim to avoid collisions
- Hash table
    - A data structure that maps keys to values
    - Provides O(!0 access times on average
    - Index into a hash table is calculated by a hash function
- Bloom filters
    - Probabilistic data structure that test whether an element is a member of a set
    - The element is either not in the set or it may be in the set

---

5/26/21

Regular Expressions

- Also known as rex or regexp
- A series of characters that defines a search pattern
- Used by databases, websites, vim, and command line programs such as grep and awk
- Lexical analysis in a compiler

Basic Regex Syntax

-

Expression	What Does It Match?	Expression	Matches (in red)
a	Matches the character itself	a	aab Santa Cruz
.	Matches any character	.ed	jumped flexed
+	Matches one or more occurrences	o+d	Food Rode
*	Matches zero or more occurrences	u*int	uint32_t int8_t
?	One or zero occurrence of a one-character expression	an?	an apple

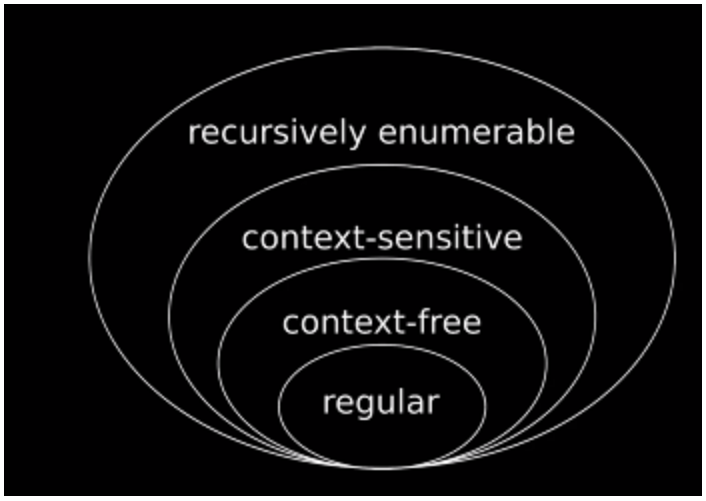
Expression	What Does It Match?	Expression	Matches (in red)
[ ]	Any character enclosed	[aeiou]	Computer
[a-z]	Any lower case letter between a-z	[a-e]	Computer
( )	Group expressions	(pre)*y	Osprey Happy
{}	Number of times an expression is matched	[a-z]{3}	and Computer computer

Electronic mail verification

- Pattern: $text_1$@$text_2$.$text_3$
- Identify subsections
  - $Expression_1$: [A-Za-z0-9]+
  - @
  - $Expression_2$: [A-Za-z0-9]+
  - $Expression_3$: \.[A-Za-z]{3}
- Combine it: [A-Za-z0-9]+@[A-Za-z0-9]+\.[A-Za-z]{3}

```
-bash-4.2$ grep -E "[A-Za-z0-9]+@[A-Za-z0-9]+\.[A-Za-z]{3}" contacts
Person1: person1@me.com
Person2: person2@you.com
Person3: person3@us.com
```

Theory of finite automata
Types of automata



- Deterministic finite automata(DFA)
  Also called ignite state automata
- Single transition per letter
- Recognize type 3 grammars recognize regular expressions

Non-deterministic finite automata(NFA)

- Have many transitions per letter
- Have transition for the empty string
- Equal in power to DFA

Push down automata

- We have a stack
- Recognize type 2 grammars
- Recognize contex-tfree languages

Linear bounded Turing Machine

- We have a linearly bounded tape
- Recognize type 1 grammars
- Recognize context-sensitive languages

Turing machine

- We have an infinite tape
- Recognize type 0 grammars
- Can compute any computable function

How are automata theory and regular expressions connected

- Finite automata
  - Abstract machines that recognize patterns such as in strings

- Finite automata
  - Abstract machines that recognize patterns such as in strings
- Regular Expressions
  - Generate patterns of strings
  - An algebraic formula

Deterministic finite automata (DFA)

- Math model of a machine that accept a particular set of words over some alphabet
- The set of words is also called the language
- A given string will always produce the same result
- A ADFA is a quintuple
  - A DFA is a quintuple $<\Sigma, S, S_0, \delta, F>$
    - Σ – Input alphabet
    - S – Finite nonempty set of states
    - S_0 – Start state
    - δ – State transition function
    - F – Final state

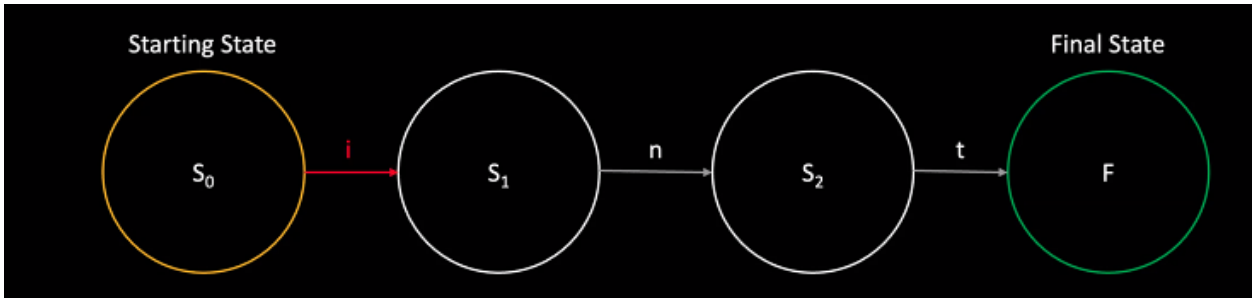
Letters and strings

- $a \in \Sigma$, we call a a *letter*
- $x \in \Sigma^*$, we call x a *string*
- $\lambda \in \Sigma^0$ is the empty string

- λ is a string
- If x is a string, then $a \cdot x$ is a string
- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$

DFA example

- Regex: “int”
- Input: int

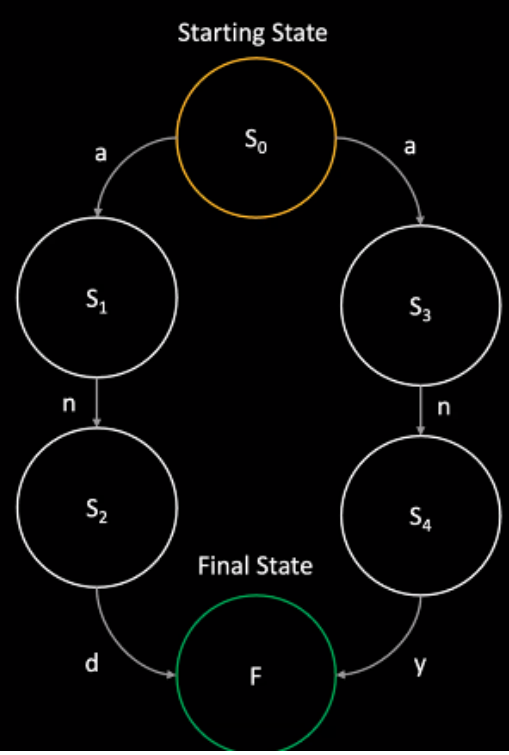


Nondeterministic Finite Automata (NFA)

- A NFA is a quintuple $<\Sigma, S, S_0, \delta, F>$
  - Σ – Input alphabet
  - S – Finite nonempty set of states
  - S_0 – Start state
  - δ – State transition function
  - F – Set of final states
- State transitions are not uniquely determined by the current state and input
- Every NFA can be written as a DFA
  - Exponentially large

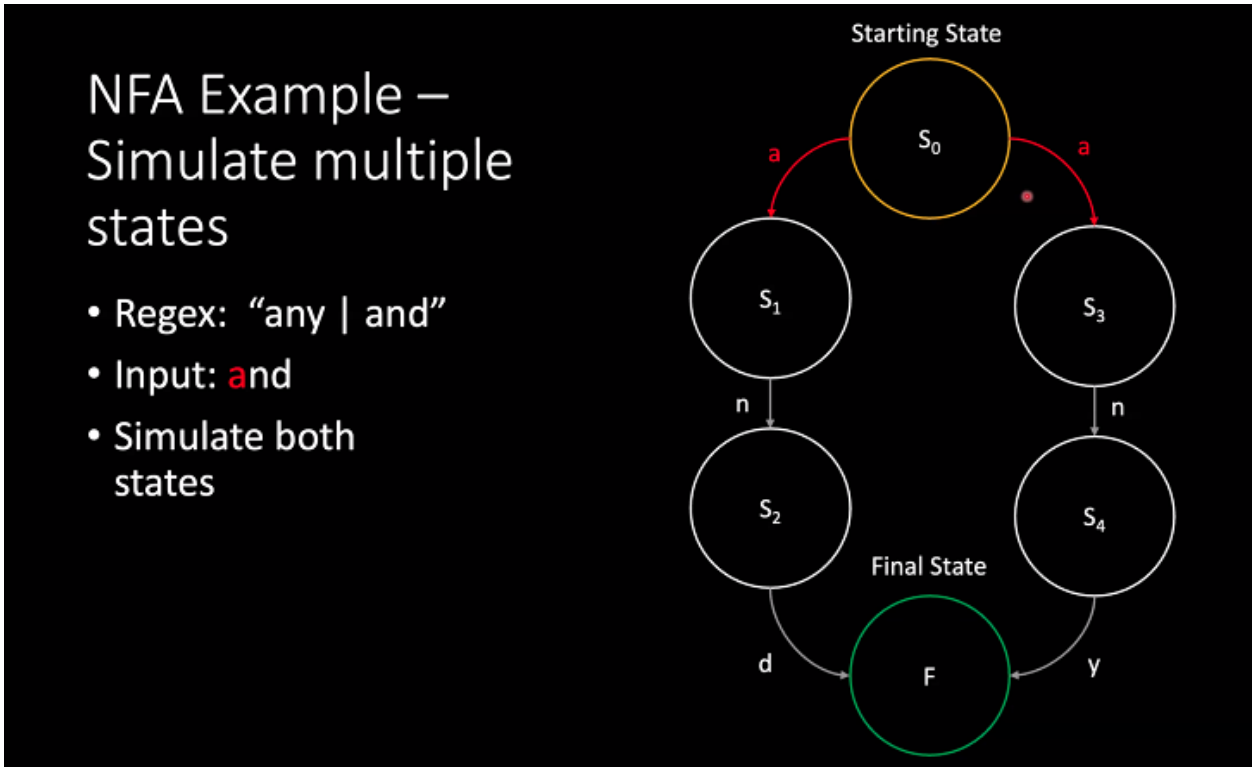
NFA example

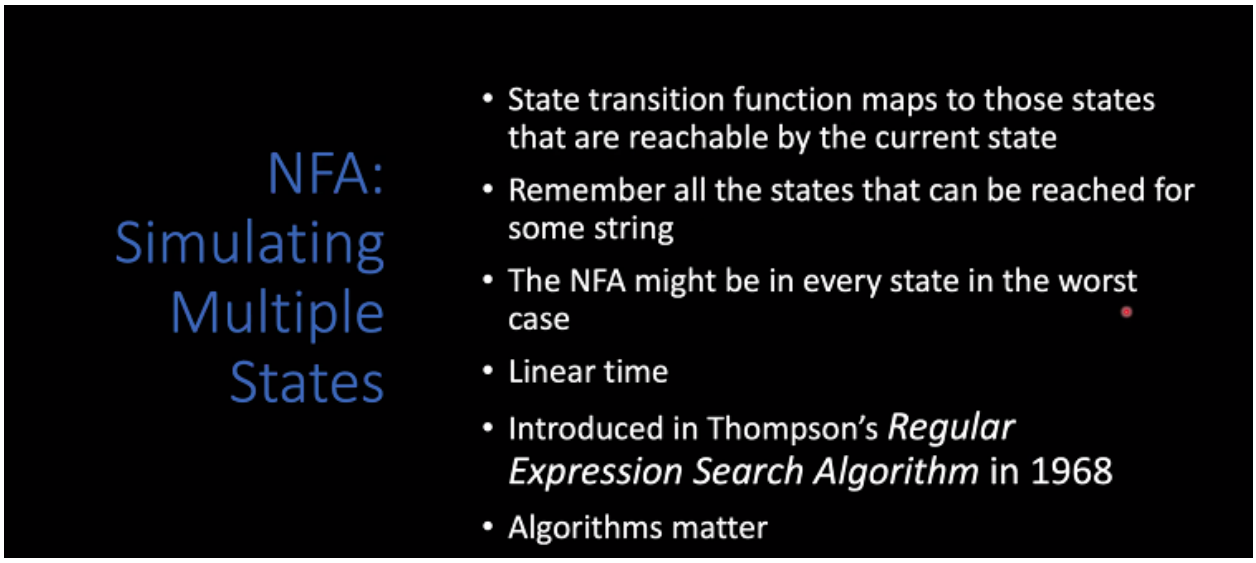
- Regex: "any | and"
- Input: and
- Must "guess" in the beginning



NFA: Backtracking

- Backtracking is an expensive recursive operation
- Must explore all possible path in the worst case
- Number of path scan grow exponentially
- Can we do better





The regex package

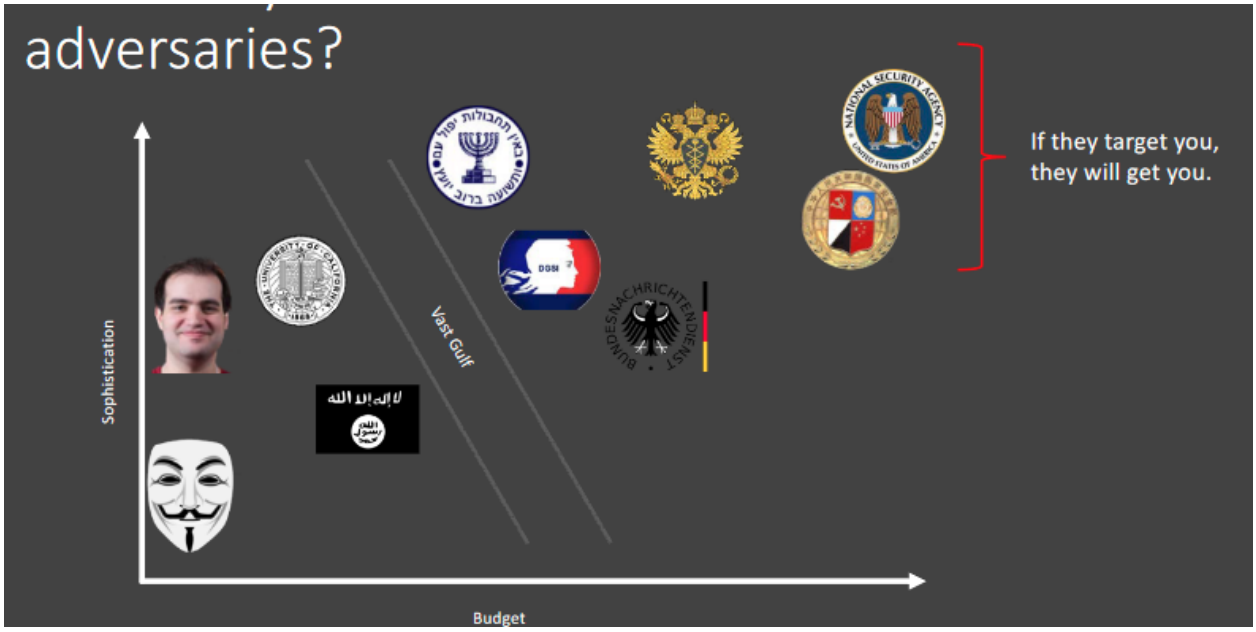
5/28/21
Security and cryptography

Security threats

- For the fun of it
- Financial gain
- Commercial advantage
- Espionage terrorism
-

Goal	Threat	
Data *confidentiality*	Exposure of data	Cryptography
Data *integrity*	Tampering with data	Cryptography
System *availability*	Denial of service	Engineering

Who are your adversaries?



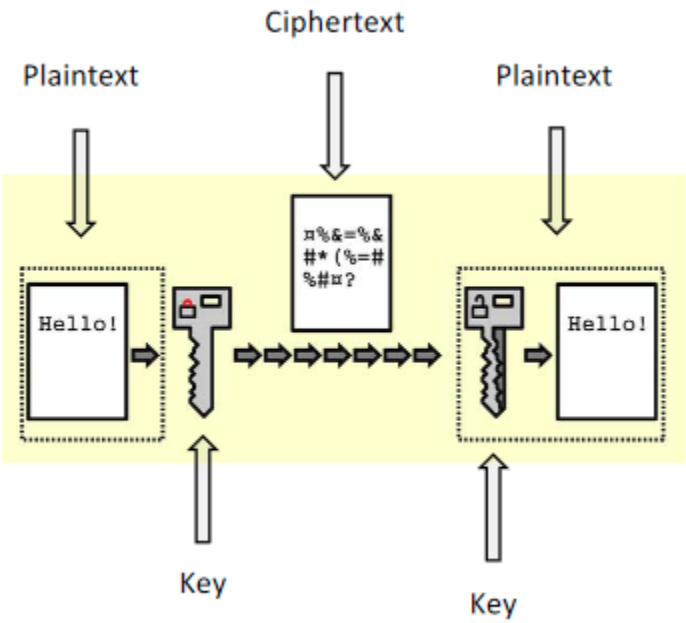
First you must prove who you are

- Something you know

- Something you have
- Something you are

Cryptography

- Goal: render a message incomprehensible to all except the intended recipient
- Requirement: use a well-known algorithm to encrypt
- But why?
    - Relying upon the secrecy of the algorithm is a very bad idea
        - German enigma
        - Japanese purple
- Algorithm has two inputs: data and key
    - Key is known only to authorized users

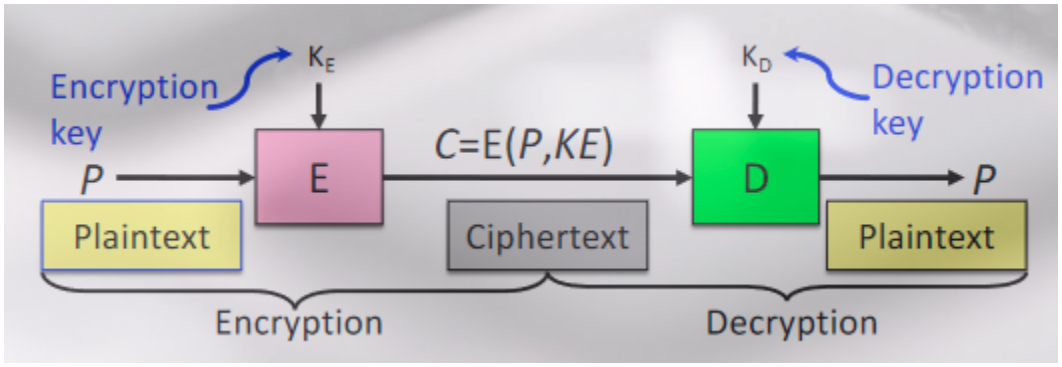


Kerckhoff's desideratum

- The system must be practically, if not mathematically, indecipherable
- It should not require secrecy, and it should not be a problem if it falls into enemy hands
- It must be possible to communicate and remember the key without using written notes, and correspondents must be able to change or modify it at will;
- It must be applicable to telegraph communications
- It must be portable, and should not require several persons to handle or operate
- Lastly, given the circumstances in which it is to be used, the system must be easy to use and should not be stressful to use or require its users to know and compu with a long list of rules

Cryptography basics

- Algorithms (E.D) are publicly known
- Keys (Ke,Kd) represent a shared secret

- The ciphertext should be the only information that's available to the world
- The plaintext is known only to the people with the keys
- 

Historical cryptography
- Cryptography has existed since there has been writing

Writing is a cipher to the illiterate and was once considered magical

Typically involved one or more of the following
- Substituting one symbol for another
- Rearranging the order of the symbols, or
- Using numbers to reference words in a known text

An early popular example is The Gold Bug by Edgar Allan Poe

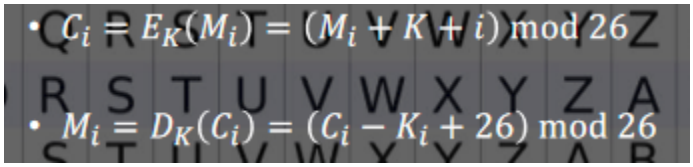
Secret Key Encryption
- Also called symmetric-key encryption
- Monoalphabetic substitution
    - Each letter replaced by different letter
- Vigenere cipher (polyalphabetic)
-

- These are easy to break
    - Given the encryption key, easy to generate the decryption key

Caesar cipher
- Reputed to have been used during the Gallic Wars
- Replaces each letter with the letter three positions down, thus
    - It uses modular arithmetic
- Why did they use they use three
    - Perhaps ancient romans were not as good at arithmetic as the greeks

Caesar cipher
- Traditionally, the caesar cipher replaces each letter with the letter three letter down
- This is a monoalphabetic substitution cipher

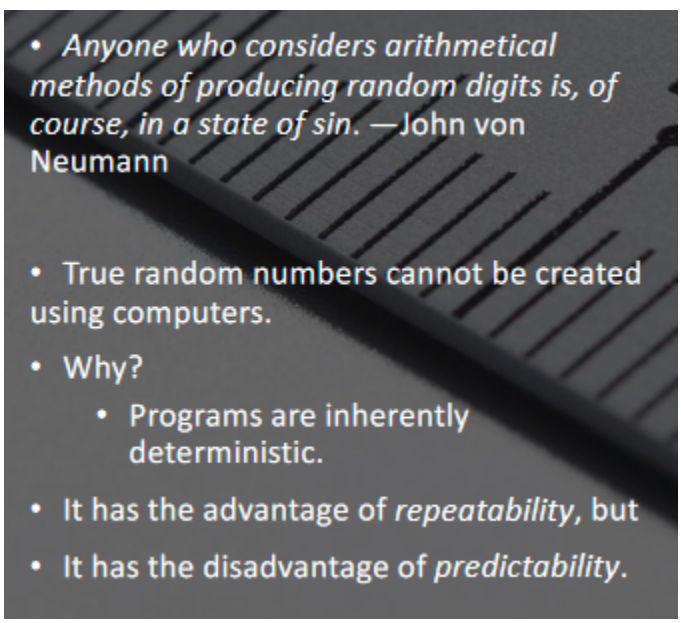
Vigenere cipher

- In this cipher, the key is a word and each letter is used to shift a letter of the clear text by its ordinal number positions
- To encrypt, a table of alphabets can be used, termed a tabula recta, vigenere square or vigenere table
- Algebraically
- 

Unbreakable codes

- There is such a thing as an unbreakable code: it's called the one-time pad
  - Use a truly random key as long as the message to be encoded
    - XOR the message with the key a bit at a time
- Code is unbreakable because
  - Key could be any string of bits
  - The message could be any message given the appropriate key
- Difficulty: distributing the key is as hard as distributing message
  - May be easier because of timing
- Difficulty: generating truly random bits
  - What does it mean to be random

Random numbers



Modern cryptography

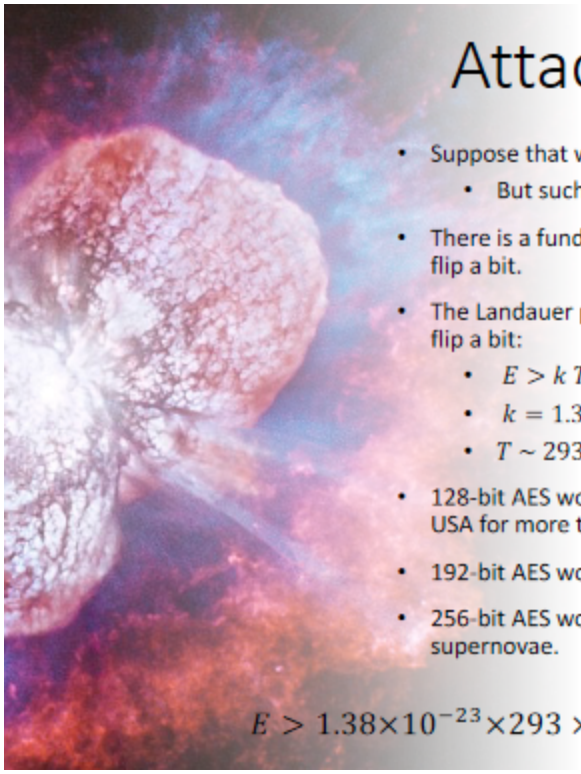
Cryptographic hashing

- A function such that:
  - Given formula for f(X), easy to evaluate y=f(x)

    - It is difficult to find collisions: two values of the same hash function
        - Weak collision resistance: given y, hard to find x such that f(x)=y
        - Strong collision resistance: hard to find x and x'=/=x such that f(x)=f(x')
- Often these operate similar to encryption algorithms
    - Produce fixed-length output rather than variable length output
    - Changing one bit of input changes 50% of the output bits
- Common algorithms include
    -
        - MD5: 128-bit result – *Broken!*
        - SHA-1: 160-bit result – *Broken!*
        - SHA-3: 244–512-bit result

Modern encryption algorithms

- Data Encryption Standard (DES) (1977)
    - Uses 56-bit keys
    - Same key is used to encrypt & decrypt
    - Needed to try 2^{55} different keys, on average
        - But… Modern computers can try millions of keys per second with special hardware
- Current algorithms (AES) use at least 128-bit keys
    - Adding one bit to the key makes it twice as hard to guess
    - Must try 2^{127} keys, on average, to find the right one
    - At 10^{15} keys per second, this would require over 10^{21} seconds, or 1,000 billion years!
    - Modern encryption isn't usually broken by brute force…



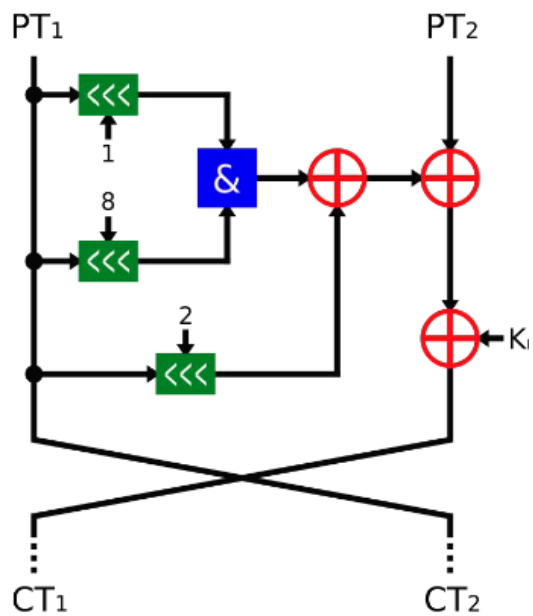
Attacking AES

- Suppose that we have an infinity fast computer,
  - But such a computer must still consume energy.
- There is a fundamental limit on the energy required to flip a bit.
- The Landauer principle says that the energy E required to flip a bit:
  - $E > k\,T\,\ln 2$ Joules
  - $k = 1.38 \times 10^{-23}$
  - $T \sim 293$
- 128-bit AES would require all the electric power of the USA for more than a year.
- 192-bit AES would require 1000 times annual solar flux.
- 256-bit AES would require more energy than a *billion* supernovae.

$$E > 1.38\times10^{-23}\times293 \times \ln 2 \times 2^{256} \approx 3.25\times10^{56} \text{ Joules}$$

Simon

- Simon is a family of lightweight block ciphers publicly released by the National Security Agency (NSA).
- Simon is a balanced *Feistel* cipher with an n-bit word, and therefore the block length is $2n$.
- As of 2018, no successful attack on full-round Simon of any variant is known.



Public-key Cryptography

- Instead of using a single shared secret, keys come in pairs:
  - The *public key* K_U.
  - The *private key* K_R.
  - $K_U \neq K_R$.
- These keys are typically—but not always—inverses of one another.
- Encryption & decryption are the *same algorithm*, so, typically:
  - $E(K_U, E(K_R, M)) = E(K_R, E(K_U, M)) = M$
- Public key cryptography is usually *slow*.
- Typically used for:
  - Encrypting small amounts of data, or
  - Establishing a shared key for symmetric encryption algorithms.
- Currently, the most popular method, RSA, involves *prime rings* and *exponentiation*.
  - Security relies on the *difficulty of factoring large composites*.
- Other methods involve *discrete logarithms* and *elliptic curves*.

Attacking RSA

- Requires factoring n in order to find $\varphi(n) = p \times q$.
- Factoring of large composites is believed to be hard.
- The best *known* algorithm is the Generalized Number Field Sieve:
  - $O(e^{\left(\sqrt[3]{\frac{64}{3}}+o(1)\right)(\ln n)^{\frac{1}{3}}(\ln \ln n)^{\frac{2}{3}}})$

El Gamal Cryptosystem

- Taher El Gamal was a student of Martin Helman.
  - He invented this public key encryption system when he was in graduate school.
- He is considered to be the father of SSL.
- His cryptosystem became the basis for the Digital Signature Algorithm (DSA) adopted by National Institute of Standards and Technology (NIST) as the Digital Signature Standard (DSS).

El Gamal Key Generation

1. Generate an efficient description of a cyclic group G, of order q, with generator g.
2. Choose an integer x randomly from $\{1, \dots, q-1\}$.
3. Compute $h = g^x$.
4. The *public key* consists of the values (G, q, g, h).
5. Alice publishes this public key and retains x as her private key.

El Gamal Encryption

A second party, Bob, encrypts a message M to Alice under her public key (G, q, g, h) as follows:

1. Map the message M to an element m of G using a reversible mapping function.
2. Choose an integer y randomly from $\{1, \ldots, q-1\}$.
3. Compute $s = h^y$. This is the *shared secret*.
4. Compute $c_1 = g^y$.
5. Compute $c_2 = m \times s$.
6. Bob sends the ciphertext (c_1, c_2) to Alice.

El Gamal Decryption

Alice decrypts a ciphertext (c_1, c_2) with her private key x as follows:

1. Compute $s = c_1^x$.
   - Since $c_1 = g^y$, $c_1^x = g^{xy} = h^y$ and it is the same shared secret that was used by Bob in encryption.
2. Compute s^{-1}, the inverse of s in the group G.
   - The modular multiplicative inverse can be computed using the *Extended Euclidean Algorithm*.
   - $s \times c_1^{q-x} = g^{xy} \times g^{(q-x)y} = (g^q)^y = e\text{^}y = e$.
3. Compute $m = c_2 \times s^{-1}$, because $c_2 = m \times s$:
   - $c_2 \times s^{-1} = (m \times s) \times s^{-1} = m \times e = m$.
4. Map m back to the plaintext message M.